1. University Scholarship Allocation

1. Problem statement

Evaluate student applications using aggregate marks, subject-wise minimum marks, attendance, and family income. List eligible students in rank order and resolve ties using aggregate, attendance, then lower family income.

2. Objectives

- Validate each applicant's marks and eligibility conditions.
- Calculate aggregate percentage and apply all eligibility rules.
- Rank eligible applicants using predefined tie-break rules.

3. Assumptions

- Each student has 3 subjects, each scored out of 100.
- Eligibility requires aggregate >= 80%, every subject >= 60, attendance >= 75%, and annual family income <= ₹300000.
- Tie-break order: higher aggregate, higher attendance, lower income.

4. Algorithm

1. Read the number of applicants.

2. For each applicant, read name, three subject marks, attendance, and annual family income.

3. Calculate aggregate percentage and check all eligibility rules.

4. Store eligible applicants as tuples.

5. Sort eligible records by aggregate descending, attendance descending, income ascending.

6. Display eligible ranks and ineligible status.

5. Flowchart

START → Read applicant count → Repeat for each applicant → Read marks, attendance, income → Calculate aggregate → Check all criteria → Eligible? –Yes→ Store record / No→ Mark ineligible → More applicants? –Yes→ Repeat / No→ Sort eligible list → Display rank list → END

6. Pseudocode

```
BEGIN
 INPUT n
 eligible = empty list
 REPEAT n times
   INPUT name, m1, m2, m3, attendance, income
   aggregate = (m1+m2+m3)/3
   IF aggregate >= 80 AND min(m1,m2,m3) >= 60 AND attendance >= 75 AND income <= 300000
      APPEND (name, aggregate, attendance, income) to eligible
   ENDIF
 ENDREPEAT
 SORT eligible by aggregate descending, attendance descending, income ascending
 DISPLAY eligible with ranks
END
```

7. Python source code

```
n = int(input("Number of applicants: "))
eligible = []
ineligible = []
for i in range(n):
    print("\nApplicant", i + 1)
    name = input("Name: ")
```

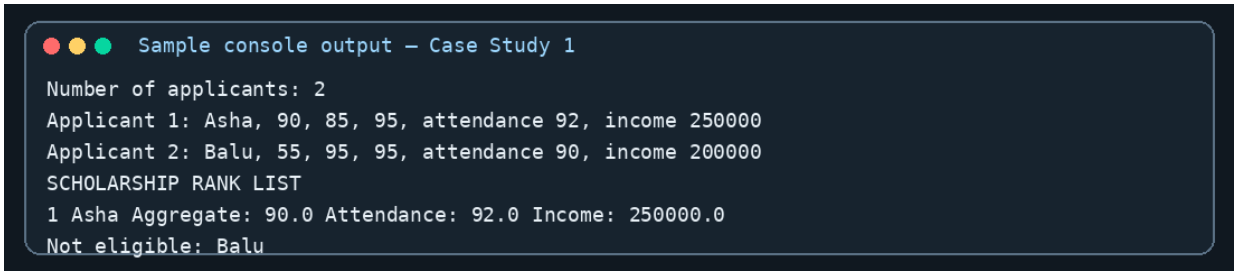
```
    m1 = float(input("Subject 1 marks: "))
    m2 = float(input("Subject 2 marks: "))
    m3 = float(input("Subject 3 marks: "))
    attendance = float(input("Attendance percentage: "))
    income = float(input("Annual family income (Rs): "))
    aggregate = (m1 + m2 + m3) / 3
    if (0 <= m1 <= 100 and 0 <= m2 <= 100 and 0 <= m3 <= 100
            and 0 <= attendance <= 100 and income >= 0
            and aggregate >= 80 and min(m1, m2, m3) >= 60
            and attendance >= 75 and income <= 300000):
        eligible.append((name, aggregate, attendance, income))
    else:
        ineligible.append(name)
eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))
print("\nSCHOLARSHIP RANK LIST")
if len(eligible) == 0:
    print("No eligible applicants.")
for rank, student in enumerate(eligible, start=1):
    name, aggregate, attendance, income = student
    print(rank, name, "Aggregate:", round(aggregate, 2),
          "Attendance:", attendance, "Income:", income)
print("Not eligible:", ", ".join(ineligible) if ineligible else "None")
```

8. Test cases

Input / condition	Expected result
Asha, marks 90,85,95; attendance 92; income 250000	Eligible; rank based on score
Balu, marks 55,95,95; attendance 90; income 200000	Ineligible: one subject below 60
Chitra, marks 80,80,80; attendance 70; income 200000	Ineligible: attendance below 75

9. Sample output screenshot

Illustrative console output for a representative test:



10. Conclusion

The program applies all scholarship rules consistently and ranks eligible students transparently using the stated tie-break order.

2. Progressive Electricity Billing

1. Problem statement

Calculate monthly household electricity bills using progressive consumption slabs, fixed charge, surcharge, and rebate. Compare bills across months and flag unusually large increases.

2. Objectives

- Calculate energy charges by slab.
- Add fixed charges and surcharge, then apply eligible rebate.
- Compare consecutive monthly bills and flag increases above a defined percentage.

3. Assumptions

- Slabs: first 100 units at ₹2/unit, next 100 at ₹3/unit, above 200 at ₹5/unit.
- Fixed charge ₹50; surcharge 5% of energy charge; rebate ₹40 when usage <= 100 units.
- An increase greater than 25% over the previous month is unusual.

4. Algorithm

1. Read number of months and each month's units.

2. Validate units are non-negative.

3. Calculate slab-wise energy charge.

4. Add fixed charge and surcharge; apply rebate if eligible.

5. Store month, units, and bill in a list.

6. Compare each bill with the previous month's bill and display increases over 25%.

5. Flowchart

START → Read month count → Read units → Validate → Calculate slab charge → Add fixed charge and surcharge → Apply rebate if eligible → Store bill → More months? → Compare consecutive bills → Flag increase >25% → Display bills → END

6. Pseudocode

```
BEGIN
 INPUT n
 bills = empty list
 FOR each month
   INPUT units
   IF units <= 100: energy = units*2
   ELSE IF units <= 200: energy = 100*2 + (units-100)*3
   ELSE: energy = 100*2 + 100*3 + (units-200)*5
   bill = energy + 50 + energy*0.05
   IF units <= 100: bill = bill - 40
   APPEND (month, units, bill)
 END FOR
 FOR each bill after first
   increase = (current-previous)/previous*100
   IF increase > 25: flag unusual
 DISPLAY bills and flags
END
```

7. Python source code

```
n = int(input("Number of months: "))
bills = []
for i in range(n):
    units = float(input("Units for month " + str(i + 1) + ": "))
    if units < 0:
```

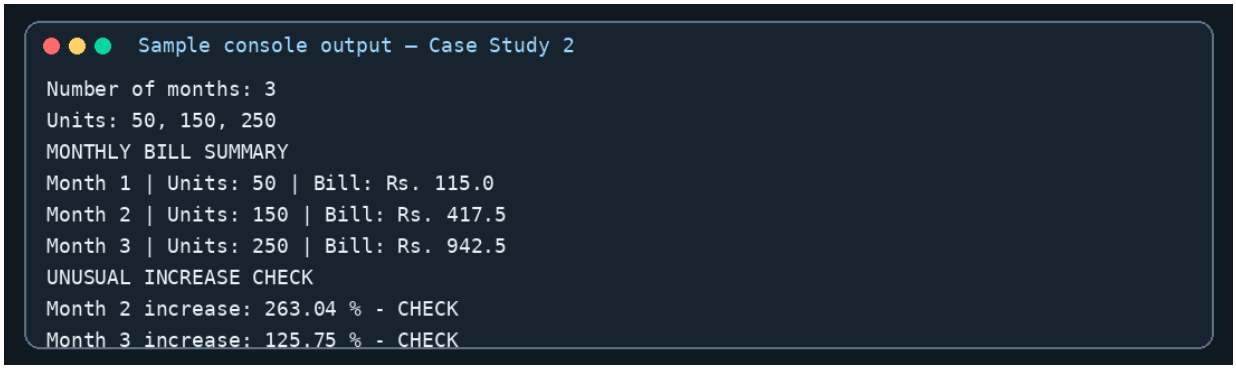
```
        print("Invalid units; enter a non-negative value.")
        continue
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5
    surcharge = energy * 0.05
    bill = energy + 50 + surcharge
    rebate = 40 if units <= 100 else 0
    bill = bill - rebate
    bills.append((i + 1, units, bill))
print("\nMONTHLY BILL SUMMARY")
for month, units, bill in bills:
    print("Month", month, "| Units:", units, "| Bill: Rs.", round(bill, 2))
print("\nUNUSUAL INCREASE CHECK")
for i in range(1, len(bills)):
    previous = bills[i - 1][2]
    current = bills[i][2]
    if previous > 0:
        increase = (current - previous) / previous * 100
        if increase > 25:
            print("Month", bills[i][0], "increase:", round(increase, 2), "% - CHECK")
        else:
            print("Month", bills[i][0], "increase:", round(increase, 2), "% - Normal")
    else:
        print("Month", bills[i][0], "- percentage change not available")
```

8. Test cases

Input / condition	Expected result
50 units	Energy 100; total bill ₹115.00 after rebate
150 units	Energy 350; total bill ₹417.50
250 units	Energy 850; total bill ₹942.50

9. Sample output screenshot

Illustrative console output for a representative test:



10. Conclusion

The program demonstrates slab-based billing and identifies month-to-month increases that may need checking.

3. Hospital Billing and Patient Classification

1. Problem statement

Calculate a patient's hospital bill from consultation, diagnostic, and treatment charges. Apply a concession based on patient category, validate inputs, and classify the account using a payment policy.

2. Objectives

- Validate charges and payment amounts.
- Apply a concession to eligible patients.
- Calculate balance and classify the payment status.

3. Assumptions

- Senior citizens receive a 10% concession on consultation and diagnostic charges only.
- No charge may be negative; payment cannot be negative.
- Status: Paid if balance is zero or less, Partially Paid if payment is greater than zero but balance remains, otherwise Unpaid.

4. Algorithm

1. Read patient name, category, and three charges.

2. Validate charges and payment.

3. Calculate gross amount.

4. If category is senior, calculate concession on consultation plus diagnostics.

5. Calculate net bill, balance, and payment status.

6. Display an itemized bill.

5. Flowchart

START → Input patient details and charges → Validate non-negative values → Calculate gross → Senior? → Apply concession if yes → Calculate net bill and balance → Classify payment status → Print invoice → END

6. Pseudocode

```
BEGIN
 INPUT name, category, consultation, diagnostics, treatment, payment
 IF any amount < 0: DISPLAY invalid and stop
 gross = consultation + diagnostics + treatment
 IF category = senior: concession = 0.10*(consultation+diagnostics) ELSE concession=0
 net = gross - concession
 balance = net - payment
 IF balance <= 0: status=Paid
 ELSE IF payment > 0: status=Partially Paid
 ELSE: status=Unpaid
 DISPLAY itemized bill
END
```

7. Python source code

```
name = input("Patient name: ")
category = input("Category (regular/senior): ").lower()
consultation = float(input("Consultation charge: Rs. "))
diagnostics = float(input("Diagnostic charges: Rs. "))
treatment = float(input("Treatment charges: Rs. "))
payment = float(input("Amount paid: Rs. "))
if min(consultation, diagnostics, treatment, payment) < 0:
    print("Invalid bill: charges and payment cannot be negative.")
else:
    gross = consultation + diagnostics + treatment
    if category == "senior":
```

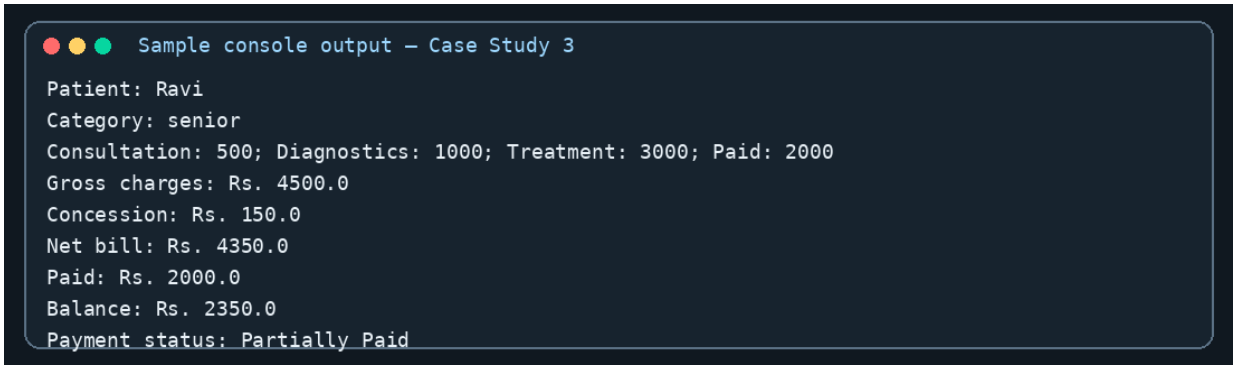
```
    concession = (consultation + diagnostics) * 0.10
else:
    concession = 0
net_bill = gross - concession
balance = net_bill - payment
if balance <= 0:
    status = "Paid"
    balance = 0
elif payment > 0:
    status = "Partially Paid"
else:
    status = "Unpaid"
print("\n------ HOSPITAL BILL ------")
print("Patient:", name)
print("Gross charges: Rs.", round(gross, 2))
print("Concession: Rs.", round(concession, 2))
print("Net bill: Rs.", round(net_bill, 2))
print("Paid: Rs.", round(payment, 2))
print("Balance: Rs.", round(balance, 2))
print("Payment status:", status)
```

8. Test cases

Input / condition	Expected result
Senior: consultation 500, diagnostics 1000, treatment 3000, paid 2000	Gross 4500; concession 150; net 4350; balance 2350; Partially Paid
Regular: 500, 1000, 3000, paid 4500	Net 4500; balance 0; Paid
Negative diagnostic charge	Invalid bill

9. Sample output screenshot

Illustrative console output for a representative test:



10. Conclusion

The billing program validates amounts, applies a clearly defined concession, and reports the remaining balance and payment class.

4. Employee Payroll and Deduction Analysis

1. Problem statement

Compute gross salary from basic pay, allowances, overtime, and incentives; calculate progressive deductions and produce a salary statement.

2. Objectives

- Calculate allowances, overtime pay, and gross salary.
- Apply progressive income deduction bands and a fixed welfare deduction.
- Display net salary in a formatted statement.

3. Assumptions

- HRA is 20% of basic pay and other allowance is 10% of basic pay.
- Overtime is paid at 1.5 times the hourly rate; hourly rate = basic pay / 160.
- Progressive deduction: 0% on first ₹20000, 5% on next ₹20000, and 10% above ₹40000 of gross salary.
- Welfare deduction is ₹200.

4. Algorithm

1. Read employee name, basic pay, overtime hours, and incentive.

2. Validate non-negative values.

3. Calculate allowances, overtime pay, and gross salary.

4. Calculate deduction progressively across salary bands.

5. Add welfare deduction and calculate net salary.

6. Print salary statement.

5. Flowchart

START → Read employee inputs → Validate → Calculate HRA, allowance, overtime → Calculate gross salary → Calculate progressive deduction → Add welfare deduction → Calculate net salary → Print statement → END

6. Pseudocode

```
BEGIN
 INPUT name, basic, overtime_hours, incentive
 IF any input < 0: DISPLAY invalid
 HRA = basic*0.20; other = basic*0.10
 overtime = basic/160*1.5*overtime_hours
 gross = basic+HRA+other+overtime+incentive
 deduction = 0
 IF gross > 20000: deduction += min(gross,40000)-20000 times 0.05
 IF gross > 40000: deduction += (gross-40000)*0.10
 welfare = 200; net = gross-deduction-welfare
 DISPLAY salary statement
END
```

7. Python source code

```
name = input("Employee name: ")
basic = float(input("Basic pay (Rs): "))
overtime_hours = float(input("Overtime hours: "))
incentive = float(input("Incentive (Rs): "))
if min(basic, overtime_hours, incentive) < 0:
    print("Invalid input: values cannot be negative.")
else:
    hra = basic * 0.20
    other_allowance = basic * 0.10
    overtime_pay = (basic / 160) * 1.5 * overtime_hours
```

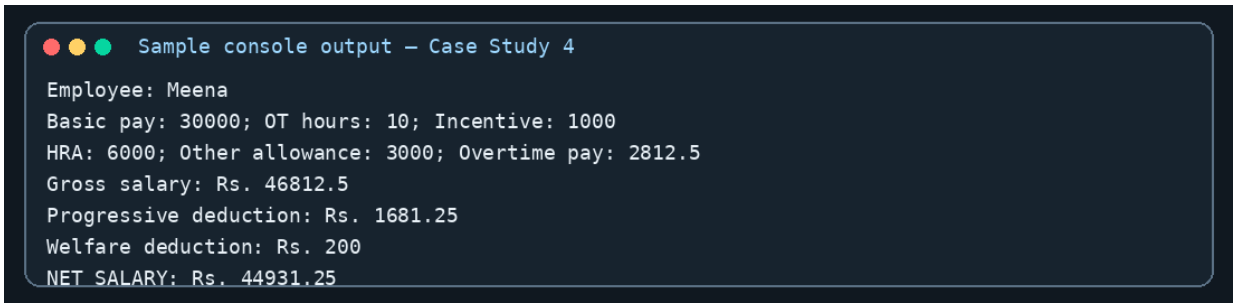
```
gross = basic + hra + other_allowance + overtime_pay + incentive
deduction = 0
if gross > 20000:
    deduction += (min(gross, 40000) - 20000) * 0.05
if gross > 40000:
    deduction += (gross - 40000) * 0.10
welfare = 200
net = gross - deduction - welfare
print("\n========== SALARY STATEMENT ==========")
print("Employee:", name)
print("Basic pay:       Rs.", round(basic, 2))
print("HRA (20%):       Rs.", round(hra, 2))
print("Other allowance: Rs.", round(other_allowance, 2))
print("Overtime pay:    Rs.", round(overtime_pay, 2))
print("Incentive:       Rs.", round(incentive, 2))
print("Gross salary:    Rs.", round(gross, 2))
print("Progressive deduction: Rs.", round(deduction, 2))
print("Welfare deduction:    Rs.", welfare)
print("NET SALARY:      Rs.", round(net, 2))
```

8. Test cases

Input / condition	Expected result
Basic 30000, OT 10 hours, incentive 1000	Gross 40312.5; progressive deduction 1015.63; net 39096.87
Basic 15000, OT 0, incentive 0	Gross 19500; progressive deduction 0; net 19300
Negative basic pay	Invalid input

9. Sample output screenshot

Illustrative console output for a representative test:



10. Conclusion

The payroll program separates gross earnings from progressive deductions and generates a readable salary statement.

5. Multi-Plan Telecommunications Cost Optimization

1. Problem statement

Compare mobile plans against a user's call, data, validity, and budget requirements. Calculate excess usage charges and choose the least-cost plan that satisfies all constraints.

2. Objectives

- Estimate each plan's total cost including excess charges.
- Exclude plans that fail validity, usage, or budget constraints.
- Select the cheapest eligible plan.

3. Assumptions

- Plans are entered by the user with price, included data in GB, included call minutes, validity days, excess data rate per GB, and excess call rate per minute.
- Usage beyond included allowance is charged proportionally.
- A plan is eligible only if validity covers required days and total cost is within budget.

4. Algorithm

1. Read required data, minutes, validity, and budget.

2. Read number of plans and each plan's details.

3. Calculate excess data and minutes.

4. Calculate total cost for each plan.

5. Keep plans that meet validity and budget requirements.

6. Sort eligible plans by total cost and display the cheapest.

5. Flowchart

START → Read user needs and budget → Read plan details → Calculate excess data/minutes → Calculate total cost → Check validity and budget → Store eligible plan → More plans? → Sort by cost → Display best plan or none → END

6. Pseudocode

```
BEGIN
 INPUT required_data, required_minutes, required_days, budget
 INPUT number of plans
 eligible = empty list
 FOR each plan
   excess_data = max(0, required_data-included_data)
   excess_minutes = max(0, required_minutes-included_minutes)
   cost = price + excess_data*data_rate + excess_minutes*call_rate
   IF validity >= required_days AND cost <= budget: APPEND (name,cost)
 END FOR
 SORT eligible by cost ascending
 DISPLAY first eligible plan or no match
END
```

7. Python source code

```
required_data = float(input("Required data (GB): "))
required_minutes = int(input("Required call minutes: "))
required_days = int(input("Required validity (days): "))
budget = float(input("Budget (Rs): "))
n = int(input("Number of plans to compare: "))
eligible = []
for i in range(n):
    print("\nPlan", i + 1)
    name = input("Plan name: ")
```

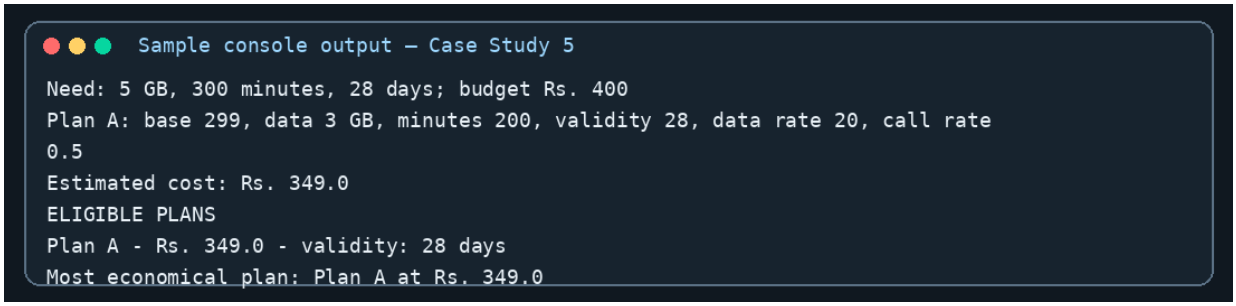
```
    price = float(input("Base price (Rs): "))
    data = float(input("Included data (GB): "))
    minutes = int(input("Included call minutes: "))
    validity = int(input("Validity days: "))
    data_rate = float(input("Excess data rate per GB (Rs): "))
    call_rate = float(input("Excess call rate per minute (Rs): "))
    if min(price, data, minutes, validity, data_rate, call_rate) < 0:
        print("Invalid plan values; skipped.")
        continue
    extra_data = max(0, required_data - data)
    extra_minutes = max(0, required_minutes - minutes)
    total = price + extra_data * data_rate + extra_minutes * call_rate
    print("Estimated cost: Rs.", round(total, 2))
    if validity >= required_days and total <= budget:
        eligible.append((name, total, validity))
eligible.sort(key=lambda x: x[1])
print("\nELIGIBLE PLANS")
if not eligible:
    print("No plan satisfies all requirements and budget.")
else:
    for plan in eligible:
        print(plan[0], "- Rs.", round(plan[1], 2), "- validity:", plan[2], "days")
    print("Most economical plan:", eligible[0][0], "at Rs.", round(eligible[0][1], 2))
```

8. Test cases

Input / condition	Expected result
Need 5 GB, 300 min, 28 days, budget 400; plan A ₹299 includes 3GB/200min with rates 20/0.5	Cost ₹349; eligible if validity >=28
Plan validity 14 days but need 28	Not eligible
All plans cost above budget	No eligible plan message

9. Sample output screenshot

Illustrative console output for a representative test:



10. Conclusion

The program compares the full estimated cost rather than only the advertised base price and chooses the cheapest plan meeting the user's needs.

6. Railway Fare and Reservation Calculator

1. Problem statement

Calculate railway fares for multiple passengers using distance slabs, travel class multipliers, age-based concessions, reservation fees, and cancellation penalties.

2. Objectives

- Compute base fare from distance and class.
- Apply age-based concession and reservation fee.
- Calculate cancellation refund using a supplied penalty rule.

3. Assumptions

- Distance fare: first 100 km at ₹1/km, next 200 km at ₹0.80/km, remaining distance at ₹0.60/km.
- Class multipliers: Sleeper 1.0, AC 2.0, First AC 3.0.
- Children below 5 travel free; ages 60+ receive 20% concession on base fare. Reservation fee ₹40 per paying passenger.
- Cancellation penalty is 25% of fare after concession plus reservation fee; for free child fare, fee still applies only if reservation fee was charged.

4. Algorithm

1. Read trip distance and passenger count.

2. For each passenger, read name, age, class, and cancellation choice.

3. Calculate distance fare progressively and multiply by class factor.

4. Apply age concession; add reservation fee unless child under 5 is free of base fare.

5. If cancelled, calculate refund after penalty; otherwise show payable fare.

6. Display each passenger's fare.

5. Flowchart

START → Read distance and passenger count → For each passenger read age/class → Calculate distance slab fare → Apply class multiplier → Apply age rule → Add reservation fee → Cancelled? → Calculate refund or payable amount → Display → END

6. Pseudocode

```
BEGIN
 INPUT distance, n
 FOR each passenger
  INPUT name, age, class, cancelled
  calculate progressive distance fare
  multiply by class factor
  IF age < 5: base=0
  ELSE IF age >= 60: base=base*0.80
  fare=base+reservation_fee
  IF cancelled: refund= fare - fare*0.25 ELSE payable=fare
  DISPLAY result
 END FOR
END
```

7. Python source code

```
distance = float(input("Travel distance (km): "))
n = int(input("Number of passengers: "))
if distance <= 0 or n <= 0:
    print("Distance and passenger count must be positive.")
else:
    for i in range(n):
```

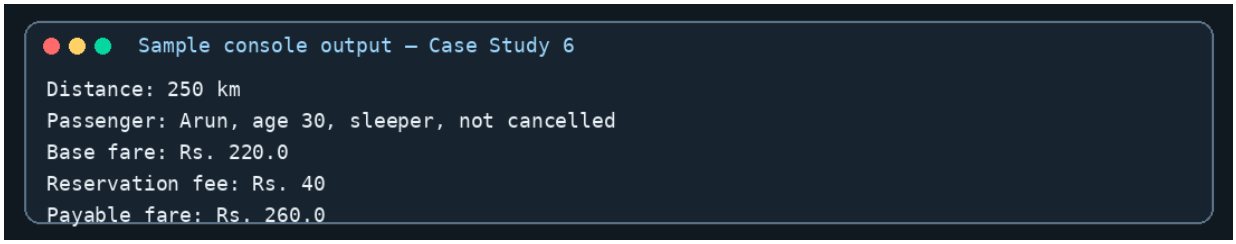
```
print("\nPassenger", i + 1)
name = input("Name: ")
age = int(input("Age: "))
travel_class = input("Class (sleeper/ac/firstac): ").lower()
cancelled = input("Cancel ticket? (yes/no): ").lower()
if travel_class == "sleeper":
    multiplier = 1
elif travel_class == "ac":
    multiplier = 2
elif travel_class == "firstac":
    multiplier = 3
else:
    print("Unknown class; passenger skipped.")
    continue
if distance <= 100:
    base = distance * 1
elif distance <= 300:
    base = 100 + (distance - 100) * 0.80
else:
    base = 100 + 200 * 0.80 + (distance - 300) * 0.60
base = base * multiplier
if age < 5:
    base = 0
elif age >= 60:
    base = base * 0.80
reservation = 40
fare = base + reservation
print("Passenger:", name, "| Fare before cancellation: Rs.", round(fare, 2))
if cancelled == "yes":
    penalty = fare * 0.25
    refund = fare - penalty
    print("Cancellation penalty: Rs.", round(penalty, 2))
    print("Refund: Rs.", round(refund, 2))
else:
    print("Payable fare: Rs.", round(fare, 2))
```

8. Test cases

Input / condition	Expected result
Distance 250 km, adult sleeper	Base = 220; fare including fee = ₹260
Distance 250 km, age 65, AC	Class base 440; after 20% concession 352; total ₹392
Cancel a ₹260 ticket	Penalty ₹65; refund ₹195

9. Sample output screenshot

Illustrative console output for a representative test:



10. Conclusion

The fare calculator combines progressive distance rates with class and age rules and provides a clear cancellation estimate.

7. Retail Billing with Multiple Discount Rules

1. Problem statement

Generate an itemized retail invoice for multiple products, quantities, category-specific taxes, threshold discounts, and a maximum concession limit.

2. Objectives

- Calculate line totals and category tax.
- Apply a threshold discount and cap the discount amount.
- Print a complete invoice with subtotal, tax, discount, and final total.

3. Assumptions

- Electronics tax is 18%, clothing tax is 5%, and groceries tax is 2%.
- A 10% discount applies when pre-tax subtotal is at least ₹5000.
- Maximum discount is ₹1000; tax is calculated on the subtotal after discount.
- Each item has non-negative price and quantity.

4. Algorithm

1. Read item count.

2. For each item, read name, category, unit price, and quantity.

3. Calculate line amount and add it to subtotal.

4. Calculate discount if subtotal meets threshold, capped at ₹1000.

5. Apply discount proportionally across subtotal for taxable amount.

6. Calculate tax by category and display itemized totals.

5. Flowchart

START → Read item count → Read item details → Validate → Calculate line amounts and subtotal → Check discount threshold → Apply discount cap → Calculate category tax → Calculate final total → Print invoice → END

6. Pseudocode

```
BEGIN
 INPUT n
 items = empty list; subtotal=0
 FOR each item: INPUT name, category, price, qty; line=price*qty; APPEND item; subtotal += line
 IF subtotal >= 5000: discount=min(subtotal*0.10,1000) ELSE discount=0
 discount_ratio=(subtotal-discount)/subtotal if subtotal>0 else 0
 FOR each item: taxable=line*discount_ratio; tax=taxable*category_rate; total_tax += tax
 final=subtotal-discount+total_tax
 DISPLAY invoice
END
```

7. Python source code

```
n = int(input("Number of products: "))
items = []
subtotal = 0
valid = True
for i in range(n):
    print("\nProduct", i + 1)
    name = input("Product name: ")
    category = input("Category (electronics/clothing/groceries): ").lower()
    price = float(input("Unit price (Rs): "))
    quantity = int(input("Quantity: "))
    if price < 0 or quantity < 0:
        print("Invalid price or quantity.")
```

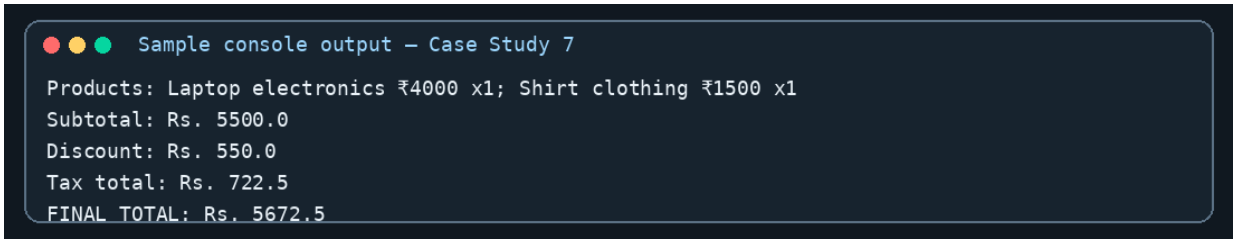
```
        valid = False
        break
    if category not in ("electronics", "clothing", "groceries"):
        print("Unknown category.")
        valid = False
        break
    line = price * quantity
    items.append((name, category, price, quantity, line))
    subtotal += line
if valid:
    discount = min(subtotal * 0.10, 1000) if subtotal >= 5000 else 0
    ratio = (subtotal - discount) / subtotal if subtotal > 0 else 0
    total_tax = 0
    print("\n---------- ITEMIZED INVOICE ----------")
    for name, category, price, quantity, line in items:
        taxable = line * ratio
        if category == "electronics":
            rate = 0.18
        elif category == "clothing":
            rate = 0.05
        else:
            rate = 0.02
        tax = taxable * rate
        total_tax += tax
        print(name, "| Qty:", quantity, "| Line:", round(line, 2),
              "| Tax:", round(tax, 2))
    final_total = subtotal - discount + total_tax
    print("Subtotal: Rs.", round(subtotal, 2))
    print("Discount: Rs.", round(discount, 2))
    print("Tax total: Rs.", round(total_tax, 2))
    print("FINAL TOTAL: Rs.", round(final_total, 2))
```

8. Test cases

Input / condition	Expected result
Electronics ₹4000 x1 and clothing ₹1500 x1	Subtotal 5500; discount 550; tax calculated on discounted item amounts
Subtotal ₹4500	No threshold discount
Quantity -1	Invalid input

9. Sample output screenshot

Illustrative console output for a representative test:



10. Conclusion

The invoice applies category taxes and a capped threshold discount, while preserving a transparent item-by-item breakdown.

8. Manufacturing Cost and Profitability Analysis

1. Problem statement

Calculate production cost from material, labour, wastage, and machine costs. Determine unit cost, selling revenue, profit margin, and whether the production run meets a target profit.

2. Objectives

- Calculate wastage-adjusted material cost and total production cost.
- Calculate revenue, profit, and profit margin.
- Compare profit with a required target.

3. Assumptions

- Wastage is entered as a percentage of material cost.
- Total cost = material cost + wastage cost + labour cost + machine cost.
- Profit target is entered as a percentage of total cost.
- All quantities and costs must be non-negative; units produced must be greater than zero.

4. Algorithm

1. Read production quantity, material cost, wastage percentage, labour cost, machine cost, and selling price per unit.

2. Validate inputs.

3. Calculate wastage cost and total cost.

4. Calculate unit cost, revenue, profit, and profit percentage.

5. Compare profit percentage with target and report result.

5. Flowchart

START → Read production and cost data → Validate → Calculate wastage cost → Calculate total cost and unit cost → Calculate revenue and profit → Compare profit percentage with target → Display analysis → END

6. Pseudocode

```
BEGIN
 INPUT units, material, wastage_percent, labour, machine, selling_price, target_percent
 IF invalid values or units <= 0: DISPLAY invalid
 wastage_cost=material*wastage_percent/100
 total_cost=material+wastage_cost+labour+machine
 revenue=units*selling_price
 profit=revenue-total_cost
 profit_percent=profit/total_cost*100 if total_cost>0 else 0
 IF profit_percent >= target_percent: target met ELSE target not met
 DISPLAY report
END
```

7. Python source code

```
units = int(input("Units produced: "))
material = float(input("Total material cost (Rs): "))
wastage_percent = float(input("Wastage percentage: "))
labour = float(input("Total labour cost (Rs): "))
machine = float(input("Machine cost (Rs): "))
selling_price = float(input("Selling price per unit (Rs): "))
target_percent = float(input("Target profit percentage: "))
if (units <= 0 or min(material, wastage_percent, labour, machine,
                      selling_price, target_percent) < 0):
    print("Invalid input. Units must be positive and other values non-negative.")
else:
    wastage_cost = material * wastage_percent / 100
    total_cost = material + wastage_cost + labour + machine
```

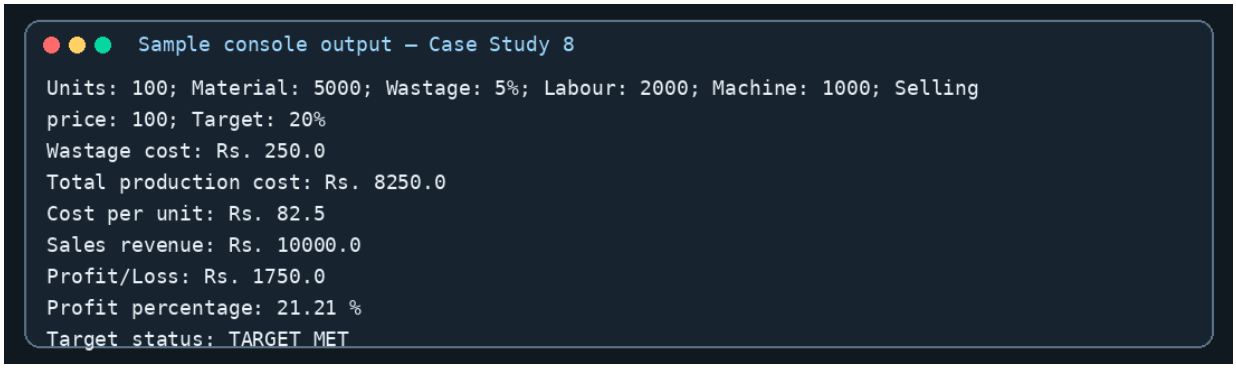
```
unit_cost = total_cost / units
revenue = units * selling_price
profit = revenue - total_cost
profit_percent = profit / total_cost * 100 if total_cost > 0 else 0
print("\n------ PRODUCTION PROFIT REPORT ------")
print("Wastage cost: Rs.", round(wastage_cost, 2))
print("Total production cost: Rs.", round(total_cost, 2))
print("Cost per unit: Rs.", round(unit_cost, 2))
print("Sales revenue: Rs.", round(revenue, 2))
print("Profit/Loss: Rs.", round(profit, 2))
print("Profit percentage: ", round(profit_percent, 2), "%")
if profit_percent >= target_percent:
    print("Target status: TARGET MET")
else:
    print("Target status: TARGET NOT MET")
```

8. Test cases

Input / condition	Expected result
100 units, material 5000, wastage 5%, labour 2000, machine 1000, sale ₹100/unit, target 20%	Wastage 250; total cost 8250; revenue 10000; profit 1750; margin 21.21%; target met
Selling price too low	Negative profit; target not met
Units 0	Invalid input

9. Sample output screenshot

Illustrative console output for a representative test:



10. Conclusion

The analysis shows how wastage and operating costs affect unit cost and helps determine whether the target profit margin is achieved.

9. Multi-Currency Travel Budget Analysis

1. Problem statement

Convert travel expenses in different currencies to Indian rupees using supplied exchange rates and conversion fees. Compare total planned spending with a budget and identify the largest expense category.

2. Objectives

- Convert foreign expenses to INR.
- Include a percentage conversion fee for each expense.
- Compare spending against budget and find the largest category.

3. Assumptions

- The user supplies the exchange rate as INR per one unit of foreign currency.
- Conversion fee is a percentage of converted INR value.
- Each expense has a category, foreign amount, currency code, exchange rate, and fee percentage.
- All amounts, rates, and fee percentages must be non-negative.

4. Algorithm

1. Read budget and number of expenses.

2. For each expense, read category, currency, amount, exchange rate, and fee rate.

3. Calculate converted value and fee.

4. Store category and total INR cost; sum all expenses.

5. Identify the largest total expense entry/category and compare total with budget.

6. Display the summary.

5. Flowchart

START → Read budget and expense count → Read each expense and exchange rate → Convert to INR → Add conversion fee → Store and total → Find largest expense → Compare total with budget → Display summary → END

6. Pseudocode

```
BEGIN
 INPUT budget, n
 expenses = empty list; total=0
 FOR each expense
  INPUT category, currency, amount, rate, fee_percent
  converted=amount*rate
  fee=converted*fee_percent/100
  final=converted+fee
  APPEND (category,currency,final); total += final
 END FOR
 FIND expense with maximum final value
 IF total <= budget: within budget ELSE over budget
 DISPLAY totals and largest expense
END
```

7. Python source code

```
budget = float(input("Travel budget in INR: "))
n = int(input("Number of expense entries: "))
expenses = []
total = 0
valid = True
for i in range(n):
    print("\nExpense", i + 1)
    category = input("Category: ")
```

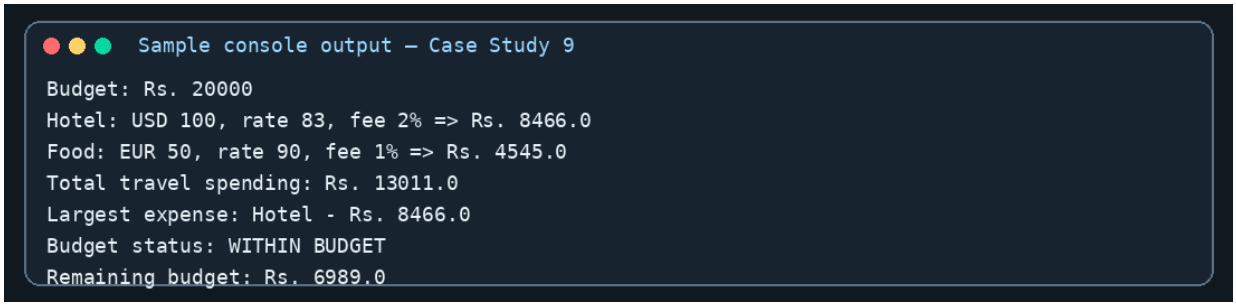
```
    currency = input("Currency code: ").upper()
    amount = float(input("Amount in foreign currency: "))
    rate = float(input("Exchange rate (INR per currency unit): "))
    fee_percent = float(input("Conversion fee percentage: "))
    if min(amount, rate, fee_percent) < 0:
        print("Invalid expense values.")
        valid = False
        break
    converted = amount * rate
    fee = converted * fee_percent / 100
    final = converted + fee
    expenses.append((category, currency, final))
    total += final
if valid:
    print("\n------ TRAVEL BUDGET SUMMARY ------")
    for category, currency, final in expenses:
        print(category, "(", currency, ") cost in INR: Rs.", round(final, 2))
    print("Total travel spending: Rs.", round(total, 2))
    if expenses:
        largest = max(expenses, key=lambda x: x[2])
        print("Largest expense:", largest[0], "- Rs.", round(largest[2], 2))
    else:
        print("No expenses entered.")
    if total <= budget:
        print("Budget status: WITHIN BUDGET")
        print("Remaining budget: Rs.", round(budget - total, 2))
    else:
        print("Budget status: OVER BUDGET")
        print("Amount over budget: Rs.", round(total - budget, 2))
```

8. Test cases

Input / condition	Expected result
Hotel USD 100 at 83 INR/USD, 2% fee	Converted 8300; fee 166; total ₹8466
Budget ₹20000, total ₹18000	Within budget; remaining ₹2000
Total above budget	Over budget and excess shown

9. Sample output screenshot

Illustrative console output for a representative test:



10. Conclusion

The program normalizes expenses into INR, includes conversion fees, and highlights both the largest expense and overall budget status.

10. Campus Event Financial Planning

1. Problem statement

Estimate campus event costs for venue, equipment, catering, transport, and certificates. Calculate participant contribution, sponsorship shortfall, and minimum registrations needed to break even.

2. Objectives

- Calculate the total event budget.
- Calculate available sponsorship and the remaining amount to fund.
- Find the minimum number of paid registrations needed to cover the remaining cost.

3. Assumptions

- Catering and certificate costs are per participant; other costs are fixed.
- Registration fee is paid by each registered participant.
- Sponsorship is a fixed contribution.
- Break-even registrations = ceiling((fixed costs - sponsorship) / (registration fee - per-person variable cost)), provided the denominator is positive.

4. Algorithm

1. Read fixed costs, per-person costs, sponsorship, registration fee, and expected participants.

2. Validate values.

3. Calculate fixed cost, variable cost per participant, and total expected cost.

4. Calculate sponsorship shortfall and expected registration income.

5. Calculate break-even registrations if each registration contributes a positive amount toward fixed costs.

6. Display the plan.

5. Flowchart

START → Read fixed costs and per-person costs → Validate → Calculate total expected cost → Subtract sponsorship → Calculate per-person contribution → Positive contribution? → Calculate break-even registrations → Compare expected registrations → Display financial plan → END

6. Pseudocode

```
BEGIN
 INPUT venue, equipment, transport, catering_per_person, certificate_per_person, sponsorship, fee,
expected_people
 fixed=venue+equipment+transport
 variable_per_person=catering_per_person+certificate_per_person
 expected_cost=fixed+variable_per_person*expected_people
 expected_income=sponsorship+fee*expected_people
 shortfall=max(0, expected_cost-expected_income)
 contribution=fee-variable_per_person
 IF contribution > 0: break_even=ceil(max(0,fixed-sponsorship)/contribution) ELSE break_even=not possible
 DISPLAY results
END
```

7. Python source code

```
venue = float(input("Venue cost (Rs): "))
equipment = float(input("Equipment cost (Rs): "))
transport = float(input("Transport cost (Rs): "))
catering = float(input("Catering cost per participant (Rs): "))
certificate = float(input("Certificate cost per participant (Rs): "))
sponsorship = float(input("Sponsorship amount (Rs): "))
fee = float(input("Registration fee per participant (Rs): "))
```

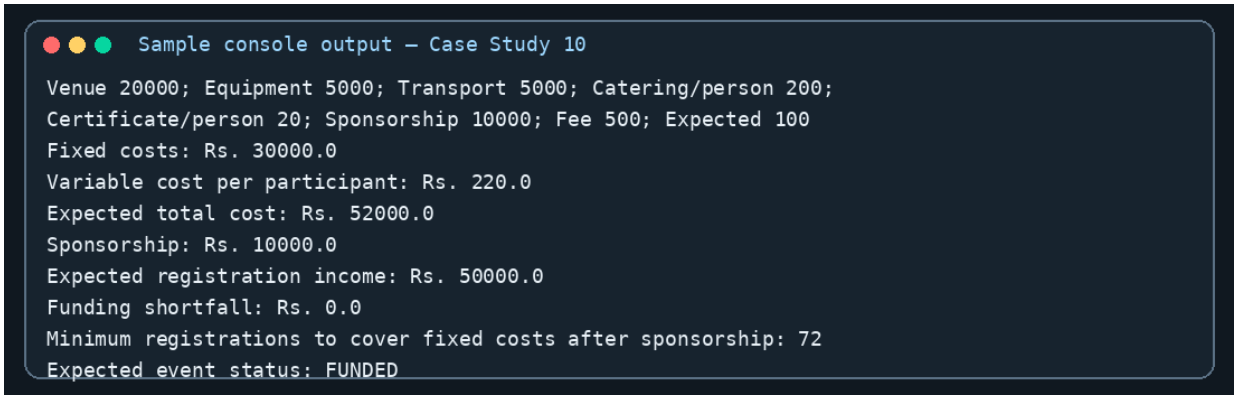
```
expected = int(input("Expected participants: "))
values = [venue, equipment, transport, catering, certificate,
          sponsorship, fee, expected]
if min(values) < 0:
    print("Invalid input: costs and counts cannot be negative.")
else:
    fixed = venue + equipment + transport
    variable_per_person = catering + certificate
    expected_cost = fixed + variable_per_person * expected
    registration_income = fee * expected
    total_funding = sponsorship + registration_income
    shortfall = max(0, expected_cost - total_funding)
    contribution = fee - variable_per_person
    print("\n------ CAMPUS EVENT FINANCIAL PLAN ------")
    print("Fixed costs: Rs.", round(fixed, 2))
    print("Variable cost per participant: Rs.", round(variable_per_person, 2))
    print("Expected total cost: Rs.", round(expected_cost, 2))
    print("Sponsorship: Rs.", round(sponsorship, 2))
    print("Expected registration income: Rs.", round(registration_income, 2))
    print("Funding shortfall: Rs.", round(shortfall, 2))
    if contribution > 0:
        amount_to_cover = max(0, fixed - sponsorship)
        break_even = int(amount_to_cover / contribution)
        if amount_to_cover % contribution != 0:
            break_even += 1
        print("Minimum registrations to cover fixed costs after sponsorship:", break_even)
    else:
        print("Break-even is not possible: registration fee must exceed variable cost per person.")
    if total_funding >= expected_cost:
        print("Expected event status: FUNDED")
    else:
        print("Expected event status: FUNDING GAP")
```

8. Test cases

Input / condition	Expected result
Fixed costs ₹30000, catering ₹200, certificate ₹20, sponsorship ₹10000, fee ₹500, expected 100	Variable ₹220/person; expected cost ₹52000; funding ₹60000; funded; break-even ceil((30000-10000)/(500-220))=72
Fee equals variable cost	Break-even not possible under formula
Negative venue cost	Invalid input

9. Sample output screenshot

Illustrative console output for a representative test:



10. Conclusion

The financial planner estimates event funding, reveals any shortfall, and computes a practical break-even registration target.